

HE-OFT: Privacy-Preserving One-Shot Federated Fine-Tuning under Homomorphic Encryption

Halil İbrahim Kanpak , Sinem Sav , Alptekin K p   

Abstract—For machine learning tasks, several parties often hold data relevant to the same task and seek to jointly train a single model without sharing or pooling their data. Federated learning (FL) enables this, but privacy risks remain due to reconstruction attacks that target shared intermediate values, i.e., model or gradients. A one-shot protocol that exchanges a single encrypted contribution removes that surface, but it still ends by handing the trained result to every participant, which is not permitted where the model itself is a regulated or proprietary asset. We present HE-OFT, the first cryptographically secure one-shot federated fine-tuning protocol in which no party receives the trained result. Each client fine-tunes a low-rank adapter and a classifier head on a frozen public backbone, retains the adapter locally, and uploads one encrypted head displacement that the server combines under multiparty CKKS and never decrypts. A quorum of clients returns only the predicted label, addressed to the party that asked. Across four text classification tasks and one vision task, HE-OFT reaches 0.61 to 0.79 accuracy where a client training alone reaches 0.20 to 0.48, gives up 0.03 to 0.14 against a disclosed model, and responds one query in 400.8 to 1468.3s on one core, or 48.9 to 176.6s with GPU acceleration, for 13.5 MiB of traffic.

Index Terms—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.

I. INTRODUCTION

Organizations adapt large public pretrained models to their own data by fine-tuning [1]. Often several organizations hold complementary data over the same task, different patient populations, fraud patterns, or customer bases, and each would obtain an enhanced model from the combination than from its own data alone [2]. However, pooling data is challenging because data protection law restricts the sharing of confidential records across institutions [3]. Three obstacles stand between that setting and a protocol these organizations could deploy.

Federated learning (FL) enables collaboration [4], and its privacy risk is concentrated at training time [5], [6]. The protocol exchanges model updates instead of data, but the updates are themselves vulnerable to privacy attacks. An adversary aggregating a client’s per-round updates can perform membership inference at 25.3% true positive rate at 0.1% false positive rate, against 0.18% from a single snapshot [7]. Two

factors therefore determine the attack surface: what a protocol reveals during training and how frequently it is revealed.

Eliminating this surface requires two measures to be taken together. Making the protocol *one-shot*, hence each client contributes only once, ensures that there is no intermediate state for anyone to observe. Encrypting the single contribution such that the only exchanged value is never available in plaintext. In this paper, *one-shot* does not mean that the parties stop communicating. Rather, it means that no intermediate training artifact is ever exposed. Prior work applies one of these measures at a time. One-shot FL [8] sends its single contribution in plaintext [9], [10], or protects it with differential privacy (DP) [11], which adds noise to the very artifact it releases. Encrypted FL runs many rounds of encrypted training or encrypted aggregation, paying the cryptographic cost on every round, and training under encryption forces polynomial replacements of every activation and a deep, repeatedly bootstrapped circuit [12]. Thus, adapting a pretrained transformer through this approach is impractical.

Applying both measures still leaves one artifact exposed, namely the model itself. One-shot protocols end by handing the final model to every participant, which rules them out when the model may not be distributed at all. A model fine-tuned on confidential records can be a regulated artifact in high-risk domains such as health, biometrics, and credit [13].

In this work, we present HE-OFT, the first one-shot federated fine-tuning protocol that enables several parties to adapt a pretrained model together and to query the result, without any of them ever holding it in plaintext. The contribution is encrypted under multiparty CKKS [14], [15], a homomorphic encryption scheme where secret key is split across the parties. The backbone remains public and in plaintext, thus the cryptographic cost does not grow with its size. At no point in the protocol is a training artifact, plaintext contribution, or plaintext model exposed.

Figure 1 shows how the shared head is built. Every client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads one encrypted head displacement weighted by the classes it holds. The server adds the ciphertexts and divides by the per-class totals under encryption, thus the shared head exists only as a ciphertext, and the server does not hold any key to decrypt it.

The querier must build its query from parts that it can run itself. Therefore, those parts must be public. The shared trained quantity therefore sits after the last nonlinearity, which, in a transformer, is a single place. Following this, we summarize our contributions below:

- **We split the trainable unit.** We federate the classifier

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Ko  University, İstanbul, T rkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, T rkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin K p   is with the Department of Computer Engineering, Ko  University, İstanbul, T rkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

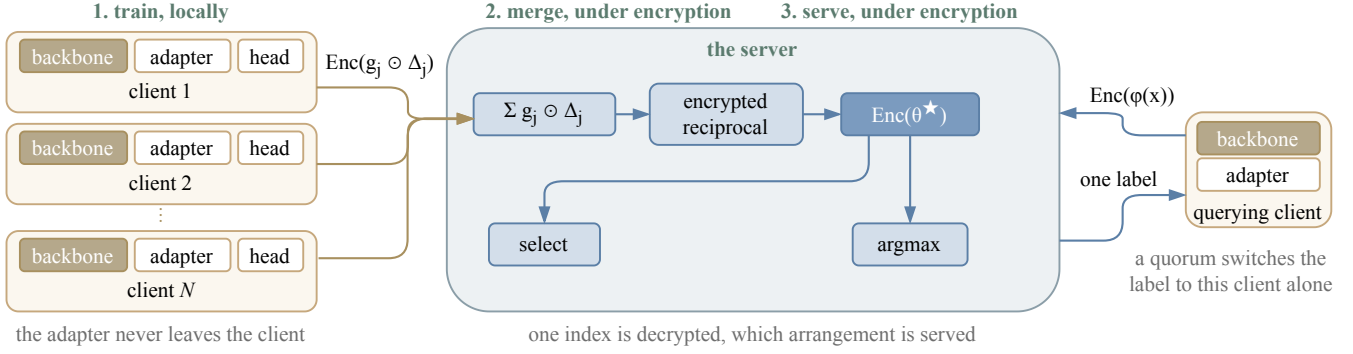


Fig. 1. The protocol in three stages. Each client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads one encrypted head displacement weighted by its per-class counts. The server adds the ciphertexts and divides by the per-class totals under encryption, holding no decryption key, so the shared head θ^* exists only as a ciphertext. The same server answers queries.

head alone. Each client keeps its adapter and never transmits it, because a client can improve its own features alone but cannot recognize a class it has never seen. The protocol forms no adapter aggregate, and a coalition of all but one client therefore has no sum from which to subtract its own contributions (Section III-B).

- **We design a serving path that never decrypts.** The querying client encrypts its own features, and the server cannot read the query. We return the label alone, and a client therefore cannot collect the per-class scores that would let it solve for the head (Section III-C).
- **We select between arrangements under encryption.** Two arrangements are servable at the same cost: the shared head over each client's own adapter and the shared head over the bare public backbone. The two fail for opposite reasons, the shared head when few clients hold a class and the personal adapter when a client's representation drifts too far. Which failure binds depends on the task and on the partition, and the federation must measure both. We show that the natural rule, each client scoring both arrangements on its own held-out data and voting, estimates the wrong quantity. Our estimator corrects it, needs no fitted threshold, and reveals only which arrangement won (Section III-D).

We evaluate HE-OFT on four text classification tasks and one vision task, with frozen RoBERTa [16] and vision transformer (ViT) [17] backbones, and we implement the cryptographic protocol in real multiparty CKKS rather than simulating it.

II. PRELIMINARIES

We denote client indices with $j \in \{1, \dots, N\}$ and class indices are $c \in \{1, \dots, C\}$. Client j holds a dataset $\mathcal{D}_j$ of size n_j and uploads a head displacement Δ_j , from which the server forms the shared head θ^* . The frozen public backbone is ϕ , while ϕ_j denotes this backbone composed with client j 's local adapter. We denote $\text{Enc}(\cdot)$ for encryption under the collective public key and t for the decryption threshold. Section II of the technical report [18] provides a comprehensive table of the notation.

A. Multiparty Homomorphic Encryption

We build on CKKS [14], which performs approximate arithmetic on packed vectors of real numbers under encryption. It is a levelled scheme such that each multiplication spends one level of a finite budget.

A single-key scheme would require one party to hold the secret key and therefore to be trusted with decryption. We use the multiparty variant [15], stated here for a t -out-of- N threshold with $2 \leq t \leq N$.

The protocol requires four operations from the multiparty scheme. Key generation produces one collective public key and the evaluation keys from the clients' shares, and never assembles the secret in one place, such that any party may compute on ciphertexts, and none may read them. Decryption takes partial shares from at least t clients, and any $t-1$ of them learn nothing about a plaintext. A key switch to a designated public key redirects a result so that one chosen party reads it and no other does. A collective refresh restores a spent level budget, and enables bootstrapping.

In the protocol, any t clients form a quorum, thus computation does not require every client to be online. Any $t-1$ clients are powerless against a ciphertext, hence $t-1$ is the corruption bound throughout Section IV.

Our implementation instantiates $t = N$, which is the setting the multiparty CKKS library provides, and the measurements of Section V-E are taken there. Every statement in Section IV holds for general t .

III. METHOD

a) *Problem Statement:* We consider N clients holding private labeled data over a common label space of C classes. Client j holds data $\mathcal{D}_j$ of size n_j . The class proportions differ across clients, and a client may hold no example of a class. The clients seek a classifier that operates across the entire label space, and they cannot pool their data to train a single model. Three requirements separate this setting from vanilla federated learning. The protocol is *one-shot*, and each client uploads its encrypted head displacement once. Confidentiality rests on cryptography, and a client's contribution is never available in plaintext to any other party. The trained classifier is never disclosed, and no party, client or server, ever holds it in plaintext.

b) *Threat Model*: The participants are N clients and a server. A static, semi-honest adversary corrupts the server together with up to $t - 1$ clients. The corrupted parties follow the protocol, pool their views, and try to infer private information from what they observe. Section IV states what each party sees and proves the guarantees.

c) *Motivation*: Optimization cannot run under encryption at an acceptable cost. All learning therefore happens in plaintext on the clients, and the server performs no learning. Models trained from different starting points do not combine linearly, and multiplicative depth dominates homomorphic cost. Every client therefore starts from one public initializer on one frozen public backbone, and the server combines what it receives by a linear combination.

Intermediate training signals are the strongest attack surface, as Section VI-A reports. The protocol is therefore one-shot, and its single artifact is a ciphertext. No single party may decrypt that artifact. The secret key is generated and held distributively under multiparty CKKS (Section II-A).

The model is never disclosed, yet it must answer queries. The client that asks a query computes the features of that query itself. It therefore runs the backbone and its own adapter in plaintext, and the shared trained quantity cannot sit inside the backbone. A server computing on ciphertexts cannot inspect a value or compare two of them, hence it cannot apply an aggregation rule that depends on the data. The clients make that choice instead (Section III-D).

A. Overview

The protocol runs in two phases. In the first, each client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads its head displacement once, weighted by its own per-class counts and encrypted. Figure 1 shows that phase. In the second, a client computes the features of its query itself, encrypts them, and sends them to the server, which applies the encrypted head and reduces the encrypted logits to the index of the largest. No party holds a key that opens the shared head alone.

B. Shared Head Construction

1) *Local Training*: Each client uses the same frozen public backbone φ , and trains a low-rank adapter A_j that adjusts the representation, and a classifier head h_j on top of it. Only the head is shared. Client j encrypts the displacement $\Delta_j = h_j - \theta_0$ of its head from the public initialiser and uploads it once. The server combines the encrypted displacements into a single shared head and never decrypts the result. The adapter is trained locally and never transmitted.

2) *Position of the Shared Map*: The client that sends an inference query must compute the features of that query itself. If the shared trained quantity were an adapter placed inside the backbone, the features would depend on it, and computing them would require evaluating the backbone with encrypted weights. The client would then have to evaluate every nonlinearity of the network homomorphically, which is the cost this design exists to avoid. The trained quantity that

is shared must therefore attach after the last nonlinearity. In a transformer, the only such point is the final linear map.

The requirement constrains the *position* of the shared map, not the task it serves. Any trained linear map placed after the final nonlinearity satisfies this requirement, provided that the querying client can compute the features fed into the map. A classifier head is one example. The vocabulary projection in an autoregressive decoder is another. Placing the map earlier forces the client either to evaluate the nonlinearities above it under encryption or to decrypt an intermediate value, which hands back the numbers the serving path withholds. We evaluate classification only.

The adapter is relocated. Each client retains its own, thus the representation still adapts to local data. The distinction follows from coverage. A client can improve its own representation alone, but it cannot learn to recognise a class it has never observed. Coverage is a property of the head, which is exactly what the federation supplies. Section V-B measures both halves.

Retaining the adapter locally has a second consequence, which is cryptographic. No aggregate of the adapters is ever formed. Section IV states this precisely.

3) *Coverage-Weighted Merge*: Let $g_j \in \mathbb{R}^C$ collect client j 's per-class example counts, $g_{j,c} = n_{j,c}$. The shared head is the coverage-weighted combination

$$\theta^* = \theta_0 + \frac{\sum_{j=1}^N g_j \odot \Delta_j}{\sum_{j=1}^N g_j}, \quad (1)$$

in which row c of the shared head is the count-weighted average of the clients' row- c displacements. Clients holding a class decide its row, and rows that no client covers remain at the public initialiser. Each client forms the products $g_j \odot \Delta_j$ locally, before encryption, thus the server only adds ciphertexts.

4) *Division by the Class Totals*: The denominator of eq. (1) is the vector of federation-wide per-class totals. It cannot be decrypted. Every client knows its own counts, hence a coalition of $N - 1$ clients that learned the totals would subtract its own and read the remaining client's class histogram exactly. The reciprocal is therefore evaluated under encryption, using an inversion circuit over the positive domain whose refreshes are collective. It is paid once, at aggregation, never per query.

5) *Aggregation Cost*: The server performs one ciphertext addition per client per ciphertext, one encrypted reciprocal, and one multiplication. The encrypted object is the head alone, whose size is set by the number of classes and the feature dimension. Communication is one upload per client and no download of model parameters at all.

C. Encrypted Inference

1) *Query Serving*: A client that wants a prediction computes the features of its query with its own backbone and adapter, encrypts them, and sends the ciphertext to the server. The server applies the encrypted shared head to the encrypted features, obtains encrypted logits, and reduces them to the index of the largest entry under encryption. Algorithm 1 states it.

Algorithm 1 Answering one query. The shared head remains a ciphertext throughout.

Require: query x at client j ; $\text{Enc}(\theta^*)$ held by the server

- 1: client j computes $\varphi_j(x)$ with its own backbone and adapter $\triangleright$ plaintext, local
- 2: client j sends $\text{Enc}(\varphi_j(x))$
- 3: **server** computes $\text{Enc}(\ell) \leftarrow \text{Enc}(\theta^*) \text{Enc}(\varphi_j(x))$ $\triangleright$ ciphertext by ciphertext
- 4: **server** reduces $\text{Enc}(\ell)$ to $\text{Enc}(\hat{y})$, $\hat{y} = \arg\max_c \ell_c$, by a tournament of homomorphic comparisons [22]
- 5: a quorum of clients key-switches $\text{Enc}(\hat{y})$ to client j 's public key
- 6: **return** $\hat{y}$, readable by client j alone

2) *Query Encryption*: The client could send the features in plaintext, which would make the head application cheaper. We encrypt them because features are invertible. A party holding $\varphi_j(x)$ and the public backbone can reconstruct an approximation of x . Sending features in plaintext would therefore hand the server the client's input, which is the thing the protocol exists to protect.

3) *Label-Only Output*: Returning logits would defeat the construction. The head is a linear map on features the client computes itself, therefore a client that reads logits for feature vectors of its choosing recovers the head by solving a linear system in as many queries as the feature dimension [19]. The same object, the final projection layer of a production transformer, has been recovered through a deployed interface for a few tens of dollars [20]. Returning only the index of the largest entry places an adversary in the hard-label setting, where functionally equivalent extraction is markedly harder [21]. It does not make extraction impossible, and Section IV states the resulting bound as a bound on queries.

4) *Decryption to the Querier*: Decryption needs a quorum, and clients other than the querier take part in answering each query. They contribute partial decryptions and read nothing, because the quorum key-switches the ciphertext to the querying client's public key rather than decrypting the label collectively.

5) *Serving Cost*: The argmax is the one operation in the protocol that costs more than one level, because it compares logits rather than adding or scaling them. Packing the logits into one ciphertext and reducing them by a tournament lowers the number of sequential comparisons from $C - 1$ to $\lceil \log_2 C \rceil$, and Section V-E reports the measured result. The server is not trusted with anything. It holds the collective public key, the evaluation keys, and ciphertexts, so it computes but cannot decrypt.

6) *Scope*: This construction serves a trained quantity that is linear in the features. Serving a trained quantity that sits inside the backbone is the problem that secure transformer inference addresses. An encrypted shared adapter composes with those systems at the cost they report, and we place that case outside our scope.

D. Arrangement Selection

1) *Encrypted Scoring*: Two arrangements are available at the same cost: the shared head over each client's own adapter, and the shared head over the bare public backbone with no

adapter at all. Which is better depends on the task, and the federation must choose without decrypting either. Each client evaluates both on data it holds back from its own training, entirely under encryption, and reports encrypted per-class counts. The server combines the counts into one encrypted score per arrangement, and the federation decrypts exactly one value, the index of the arrangement it adopts.

2) *Prior-Weighted Estimator*: Each client could instead measure both arrangements on its held-out data and vote. That procedure estimates the wrong quantity, and Section V-D shows it choosing wrongly whenever the two arrangements differ. Score both arrangements as the expected accuracy of a prediction on an example drawn from the federation's own class distribution,

$$E = \sum_{c=1}^C p_c a_c, \quad p_c = \frac{\sum_j n_{j,c}}{\sum_j n_j}, \quad (2)$$

where a_c is the accuracy on class c and p_c is that class's share of the federation's examples, taken from the per-class counts already used in eq. (1). For the shared arrangement, there is one model, hence per-class evidence from different clients concerns the same object and combines. For the personal arrangement each client's evidence concerns only its own model, and classes it does not hold contribute no evidence at all. Scoring those classes at zero makes E a lower bound for the personal arrangement. There is no fitted threshold anywhere.

3) *One Example per Class*: The estimator needs an estimate of accuracy on classes a client has not seen, and its only local proxy is accuracy on classes it has barely seen. Each client therefore reserves at least one held-out example from every class it holds before carving the remainder at random. This costs a negligible amount of training data and requires no change to the cryptographic protocol.

4) *Cost and Disclosure*: Every step is depth one except the comparisons, which are the same circuit already used for the argmax. One value is decrypted for the whole federation. The per-client per-class accuracies are not decrypted, and must not be.

IV. SECURITY

The participants are N clients and a server. A static, semi-honest adversary corrupts the server together with up to $t - 1$ clients. The corrupted parties follow the protocol, pool their views, and try to infer private information from what they observe. Section IV-C strengthens the client side of this model, and allows up to $t - 1$ clients to deviate arbitrarily while the server stays honest. Section IV-D explains why that assumption cannot be dropped.

The server sees ciphertexts under the collective public key, the public per-client sample counts, and the evaluation keys. It does not hold the decryption key. The server additionally sees encrypted query features, which it cannot read.

Three properties of the construction close the routes a coalition could take. Every adapter stays with its owner, and the protocol never forms the aggregate a coalition would subtract its own contributions from. The shared head stays

Functionality 1 HE-OFT $\mathcal{F}$

Parameters. The client count N , the decryption threshold t , the per-client query allowance Q , and the label space size C . The parties are clients $P_1, \dots, P_N$ and a server S , which is not part of $\mathcal{F}$. The functionality holds a counter q_j for each client, each initialised to 0, and a store that is initially empty and that $\mathcal{F}$ never sends to any party.

Inputs. Each client P_j inputs its dataset $\mathcal{D}_j$, and later inputs queries x . The server S inputs a selection request.

Outputs. Every party receives the selected index a^* and the phase signals. Client P_j receives one label for each of its first Q queries, and $\perp$ afterwards.

Steps.

- 1) *Training and aggregation.* On $(\text{Train}, \mathcal{D}_j)$ from every client, run the local training of Section III-B on each $\mathcal{D}_j$ in both servable arrangements and store the two shared heads θ_A^* and θ_B^* of eq. (1). Send $(\text{Done}, \text{train})$ to every party.
- 2) *Selection.* On (Select) from S , evaluate the estimator of eq. (2) on each of the two stored heads, send the index $a^* \in \{A, B\}$ of the larger to every party, and send $(\text{Done}, \text{select})$ to every party.
- 3) *Serving.* On (Query, x) from client P_j , return $\perp$ to P_j if $q_j \geq Q$. Otherwise set $q_j \leftarrow q_j + 1$, return $\hat{y} = \arg \max_c (\theta_{a^*}^* \varphi_j^{a^*}(x))_c$ to P_j alone, where $\varphi_j^A = \varphi$ is the public backbone and $\varphi_j^B = \varphi_j$ is the backbone composed with P_j 's adapter, and send $(\text{Done}, \text{query}, j)$ to S .

Leakage. $\mathcal{F}$ reveals $\mathcal{L}$ to the adversary, consisting of the public parameters, the per-client sample counts n_j , the announced index a^* , and the fact that each Done message was sent.

a ciphertext throughout, which closes the same route against it. The server inverts the per-class totals under encryption and never decrypts them. A coalition cannot recover the remaining client's class histogram.

A. The Ideal Functionality

Functionality 1 defines the ideal functionality $\mathcal{F}$ against which we measure the protocol. The server sits outside $\mathcal{F}$, and the semi-honest adversary of Section IV may corrupt it. The form of the functionality follows the secure aggregation functionality of ELSA [23], and placing the aggregator outside it follows FULLSA [24].

The sample counts sit in the leakage because the protocol uses them as public scalars. We prove security relative to both.

Three properties of $\mathcal{F}$ are what the protocol must reproduce. No party ever holds either shared head. The only value the whole federation learns is one index. A client learns one label for each query it submits, up to its allowance Q , and nothing else.

B. Security Against Semi-Honest Parties

Definition 1 (Semi-honest security). Let $\mathcal{A}$ corrupt a set of parties that may contain the server and at most $t - 1$ clients. Write $\text{view}_{\mathcal{A}}^{\Pi}(\{\mathcal{D}_j\}_{j=1}^N)$ for the corrupted parties' joint view of a protocol run on datasets $\{\mathcal{D}_j\}_{j=1}^N$. The protocol Π securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ if there is a probabilistic polynomial-time simulator $\mathcal{S}$ such that, for every $\{\mathcal{D}_j\}_{j=1}^N$,

$$\text{view}_{\mathcal{A}}^{\Pi}(\{\mathcal{D}_j\}_{j=1}^N) \stackrel{c}{\approx} \mathcal{S}(\mathcal{L}, \{\mathcal{D}_j\}_j \text{ corrupt}, \text{out}_{\mathcal{A}}),$$

where $\text{out}_{\mathcal{A}}$ collects the answers $\mathcal{F}$ returns to the corrupted clients.

Theorem 1 (Semi-honest security). *Assume the t -out-of- N threshold CKKS scheme is secure under chosen plaintext*

attack (IND-CPA) against an adversary holding $t - 1$ key shares. Then the protocol securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ against any semi-honest adversary corrupting the server and at most $t - 1$ clients, in the sense of definition 1.

Proof sketch. The simulator is given $\mathcal{L}$, the corrupted clients' datasets, and the answers the corrupted clients receive.

Setup. The simulator runs the distributed key generation with the corrupted parties, playing the honest clients on uniformly chosen shares.

Training. For each honest client, the simulator sends an encryption of zero in place of $\text{Enc}(g_j \odot \Delta_j)$, once for each of the two arrangements, and in place of $\text{Enc}(g_j)$, which the two share. The simulator knows how many ciphertexts to send, because the count follows from the public parameters. A hybrid argument replaces the honest ciphertexts one at a time, and each neighbouring pair of hybrids is indistinguishable by IND-CPA against $t - 1$ shares, the assumption of the theorem.

Selection and serving. The selection step decrypts the index a^* of the adopted arrangement. The simulator takes it from $\mathcal{L}$. For each query of a corrupted client, the simulator holds the answer in $\text{out}_{\mathcal{A}}$. Queries by honest clients are simulated as key switches of encryptions of zero.

Composing the three parts gives a view computationally indistinguishable from the real one. Section IV of the technical report [18] gives every proof in full. $\square$

C. Input Privacy Against Malicious Clients

Theorem 1 assumes every party follows the protocol. We extend the model to clients that deviate arbitrarily. We bound what a deviating coalition learns about the data of a client that follows the protocol. We do not claim correctness.

Definition 2 (Content game). The adversary $\mathcal{A}$ corrupts $t - 1$ clients. The server is honest. At least one client h stays honest.

- 1) $\mathcal{A}$ outputs two datasets D_0 and D_1 with $|D_0| = |D_1|$.
- 2) The challenger samples $b \in \{0, 1\}$ and gives D_b to client h .
- 3) The protocol runs. $\mathcal{A}$ may deviate arbitrarily in the values its clients upload, in the shares they contribute, and in the queries they ask.
- 4) $\mathcal{A}$ outputs b' . Its advantage is $|\Pr[b' = b] - \frac{1}{2}|$.

Theorem 2 (Input privacy). *Assume the conditions of theorem 1. Then the advantage of $\mathcal{A}$ in definition 2 is at most $\text{negl}(\lambda) + \delta(Q_{\text{tot}})$, where $Q_{\text{tot}} = (t - 1)Q$ and δ is the advantage available from Q_{tot} label queries to the served head.*

Proof sketch. The adversary's view consists of the key-generation transcript, the honest clients' uploaded ciphertexts, the values the server computes, and the answers to its own queries.

The first four are independent of b up to $\text{negl}(\lambda)$.

Only the answers remain. They depend on b , because θ^* depends on client h 's displacement, and the advantage they give is bounded by what Q_{tot} label queries reveal. $\square$

The bound scales with the corruption threshold. A coalition of $t - 1$ clients holds $(t - 1)Q$ queries regardless of the size of

the federation. Section V of the technical report [18] measures δ and turns it into a bound on Q .

D. Necessity of an Honest Server

Theorem 2 keeps the server honest. That assumption is necessary. Decryption is a protocol the clients run on a ciphertext the server presents. A deviating server can present a ciphertext of its own choosing, such as a row of θ^* , rather than the output of algorithm 1. The natural repair is to have an honest client refuse such a request. It cannot, and the obstacle is the encryption itself.

Proposition 1 (No plaintext gate from a ciphertext). *Let an honest client decide, by a polynomial-time predicate D on its view, whether to contribute its decryption share for a presented ciphertext. Let the rest of that view be independent of the underlying plaintext. If the scheme is IND-CPA against an adversary holding $t - 1$ key shares, then for every pair of plaintexts m_0 and m_1 ,*

$$|\Pr[D(\text{Enc}(m_0)) = 1] - \Pr[D(\text{Enc}(m_1)) = 1]| \leq \text{negl}(\lambda).$$

Proof. A gap between the two probabilities gives an IND-CPA distinguisher that holds one key share. One share is at most $t - 1$ shares. The assumed security of the scheme applies directly. $\square$

An honest client therefore releases its share for both plaintexts or for neither. No protocol of this message pattern realizes $\mathcal{F}$ against a malicious server, which is why theorem 2 places the server among the honest parties.

A mechanism checks provenance. The serving circuit is a deterministic function of the encrypted head, the encrypted query, the evaluation keys and the public parameters, and its level restoration is deterministic as well. A client can therefore recompute the prescribed output ciphertext and compare it as a string.

E. Leakage Inherent to the Functionality

$\mathcal{F}$ answers queries. Answers carry information about θ^* , and enough of them reconstruct it. Section V-F measures the rate at three to five queries per parameter. This is a property of the functionality. Any protocol that realizes $\mathcal{F}$ inherits it.

The cryptographic claim is that the protocol adds nothing to the leakage of $\mathcal{F}$. The operational claim is that $\mathcal{F}$'s own leakage is bounded by the query allowance Q . The first rests on IND-CPA. The second rests on a measurement.

V. EXPERIMENTS

A. Setup

The trainable unit is a rank-8 low-rank adapter with its down-projection frozen at a shared public initialisation [25], together with a classifier head. We use no labelled public data anywhere.

We evaluate on four text classification tasks of increasing label-space size, AG-News with 4 classes, TREC with 6, DBpedia with 14 and Banking77 with 77, using a frozen RoBERTa-base encoder [16], and on CIFAR-100 with a frozen

TABLE I
ACCURACY ON THE GLOBAL TEST SET, THREE SEEDS, $N = 10$, $\alpha = 0.1$. THE SERVABLE COLUMNS ARE THE TWO ARRANGEMENTS THE PROTOCOL ANSWERS QUERIES FROM, AND BOLD MARKS THE BETTER ONE. THE REFERENCE COLUMNS ARE NOT SERVABLE UNDER THE THREAT MODEL OF SECTION IV. ALONE IS ONE CLIENT'S OWN MODEL WITH NO FEDERATION, DISCLOSED IS THE MERGED MODEL THIS PROTOCOL DECLINES TO BUILD, AND POOLED IS CENTRALISED TRAINING ON THE UNION OF THE DATA.

Task	C	servable			reference	
		shared head	adapter	alone	disclosed	pooled
AG-News	4	0.649	0.479	0.475	0.739	0.921
TREC	6	0.607	0.429	0.400	0.711	0.967
DBpedia	14	0.789	0.754	0.451	0.925	0.988
Banking77	77	0.206	0.686	0.249	0.760	0.923
CIFAR-100	100	0.748	0.774	0.197	0.784	—

ViT-B/16 [17]. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α , the standard label-skew partition [26]. There are $N = 10$ clients, $\alpha = 0.1$, the local trajectory is 200 steps, and every number is the mean over three seeds.

We write A_{sel} for the accuracy of the servable arrangement that Section V-D selects. Three reference points appear throughout, and none of them is servable under the threat model of Section IV. We write A_{loc} for the accuracy a client obtains alone. We write A_{dis} for the accuracy of the disclosed model, in which the adapter and the head are both aggregated and decrypted. We write A_{pool} for the accuracy of one model trained on the union of the clients' data.

Two differences follow, and the paper reports both. The quantity $A_{\text{dis}} - A_{\text{sel}}$ is the price of never disclosing a model. The quantity $A_{\text{pool}} - A_{\text{dis}}$ is the price of the partition.

B. Utility of the Federated Head

Sharing only the classifier head, over representations that each client adapts locally, yields a model usable across the whole label space, and the preferable arrangement depends on the size of the label space.

Table I reports both servable arrangements and both reference points. The arrangement labelled *shared head* gives every client the same head over the bare public backbone. *Personal adapter* gives each client the shared head over its own locally adapted representation.

Three observations follow. First, the federation raises accuracy on every task. A client alone reaches 0.20 to 0.48, while the federated head reaches 0.61 to 0.79 on four of the five tasks. The gap is largest exactly where a client's own data covers least of the label space.

Second, the preferable arrangement changes with the size of the label space. On the small label spaces the shared head wins, 0.17 on AG-News and 0.18 on TREC. On the large ones the personal adapter wins. The shared head collapses to 0.206 while the personal adapter holds 0.686. The collapse is a coverage failure. With 77 classes each head row is decided by few clients or none.

Third, the two failure modes are complementary. The shared head fails when coverage is thin. The personal adapter fails

TABLE II

CIFAR-10 ON TWO PUBLISHED PARTITIONS, THREE SEEDS, ON THE WHOLE 10,000-IMAGE TEST SET. THE COLUMNS ARE THE QUANTITIES DEFINED IN SECTION V-A, ALL MEASURED ON OUR PROTOCOL AT THE PARTITION EACH PRIOR PAPER PUBLISHED.

N	α	A_{loc}	A_{sel}	A_{dis}	$A_{\text{dis}} - A_{\text{sel}}$
5	0.10	0.384	0.949	0.963	0.013
5	0.30	0.570	0.952	0.966	0.015
20	0.04	0.212	0.950	0.947	-0.003
20	0.16	0.397	0.953	0.961	0.007

when each client's representation drifts far enough that a common head no longer transfers. Neither arrangement dominates the other, so the federation must select between them.

a) *Sensitivity*: Figure 2 varies the protocol axes around the default on DBpedia. Accuracy falls with skew, from 0.970 at $\alpha = 1.0$ to 0.789 at $\alpha = 0.1$. Longer local training helps. Accuracy rises with the size of the federation, from 0.803 at $N = 10$ to 0.882 at $N = 50$.

C. Comparison with One-Shot Federated Learning

The protocol gives up no accuracy for protecting the contributions, where the differentially private one-shot methods give up an amount that grows as their budget tightens. The peer group evaluates on different tasks from ours, so we ran our protocol on theirs. We adopt two published partitions of CIFAR-10: $N = 5$ with $\alpha \in \{0.1, 0.3\}$, the configuration of [9], and $N = 20$ with $\alpha \in \{0.04, 0.16\}$, the configuration of [11].

At $N = 5$ and $\alpha = 0.1$ we measure $A_{\text{sel}} = 0.949$, against the 0.503 of [9]. We do not present that margin as a result. Those methods train a ResNet-18 from scratch, whereas every client here fine-tunes a frozen public ViT-B/16. Absolute accuracy across papers therefore measures the backbone.

What compares across papers is the accuracy each method gives up for protecting the contribution, since every paper measures that against its own unprotected baseline on its own architecture, and the backbone cancels. Read that way the peer group falls into three cases. A method either leaves the contribution unprotected, protects it with differential privacy, or protects it cryptographically.

FedAUXfdp does protect its contribution, and its cost is governed by the budget. Against its own undefended baseline of 0.752 on CIFAR-10 at $N = 20$ and $\alpha = 0.16$, the published figures give up 0.006 at $\varepsilon = 1.0$ and 0.029 at $\varepsilon = 0.5$, then 0.413 at $\varepsilon = 0.1$ and 0.626 at $\varepsilon = 0.01$ [11].

Our contribution is protected cryptographically, and that protection costs no accuracy, by construction, since the decrypted result matches the plaintext computation to the encoding precision of the scheme. Withholding the model is a separate charge, and no method above incurs it. Over table II the charge lies between -0.003 and 0.015, and over table I between 0.03 and 0.14.

One difference bounds the federation's own contribution independently of the backbone, because we measure both of its terms on the same backbone. At $N = 5$ and $\alpha = 0.1$ we measure $A_{\text{loc}} = 0.384$ against $A_{\text{sel}} = 0.949$. That difference is what the protocol supplies.

D. Accuracy of the Selection Rule

The natural procedure is for each client to measure both arrangements on data it held back and vote. It estimates the wrong quantity. The shared arrangement is one model, so pooled measurements track its global accuracy, while client j can measure the personal arrangement only on its own distribution, which is the one its own model was fitted to. The vote is therefore optimistic for the personal arrangement and picks correctly in 8 of 27 cells. The estimator of eq. (2) scores both under the federation's own class prior and picks correctly in 23, or 24 when unobserved classes are filled from each client's rarest classes rather than with zero. A wrong choice gives up 0.470 of accuracy under the vote and 0.021 under the estimator. Section V of the technical report [18] reports every cell.

E. Cryptographic Cost

The cryptographic cost of the protocol is dominated by one-time setup, and the recurring per-query traffic is a few megabytes. We do not simulate the cryptography. We implement the protocol in real multiparty CKKS on Lattigo [27], [15]. All figures use ring degree 2^{14} for the shallow operations and a deeper chain at ring degree 2^{15} for the circuits that need it. Timings are single-run wall clock on one core of a VALAR CPU node. Communication figures are exact.

a) *End-to-end query cost*: On one core a query takes 400.8 s at four classes and 1468.3 s at a hundred. The arithmetic and the key switch account for 0.24 s of that at every label-space size. Everything else is the encrypted argmax, and 97 per cent of the argmax is the server restoring levels, which it does 17 times at four classes and 62 times at a hundred. The tournament needs $\lceil \log_2 C \rceil$ rounds against the $C - 1$ of the sequential fold it replaces (fig. 2).

b) *Communication*: The protocol is one-shot in the sense of Section I. No intermediate training artifact is ever exposed. It is not silent. Key generation precedes it, selection costs at most $2NC$ encrypted comparisons, once, and serving costs traffic per query.

The per-query total is $8.5 + 0.5N$ MiB, since the quorum returns one key-switching share each. It is 11.0 MiB at $N = 5$ and 18.5 MiB at $N = 20$, and it is 6.8 MiB at ring degree 2^{14} and 27.0 MiB at 2^{16} . It does not vary with the number of classes. Setup costs 344 MiB per client, once.

c) *Correctness*: The encrypted argmax agrees with the plaintext maximum to 7.3×10^{-5} , which is far below the gap between adjacent logits, so it selects the same class. The accuracy figures of table I are therefore unaffected by the cryptography.

F. Residual Leakage

The protocol exposes no training-time artifact and no model, so the only remaining channel is the sequence of answers the served model returns, and that channel is bounded by the query allowance rather than eliminated.

Fidelity is the rate at which a copy agrees with the served model. Reaching fidelity 0.90 takes about 1.2×10^4 queries on

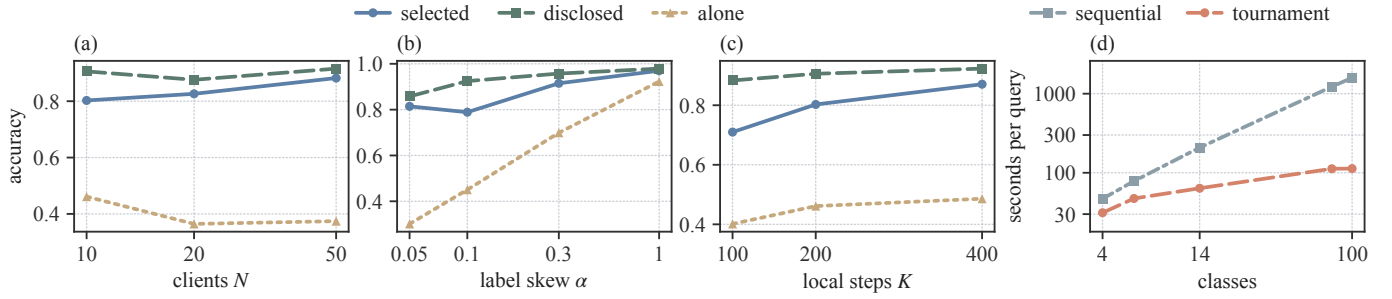


Fig. 2. The protocol along four axes. Panels (a) to (c) vary the federation size, the label skew and the local trajectory on DBpedia around $N = 10$, $\alpha = 0.1$ and 200 steps. Panel (d) gives the cost of one encrypted argmax against the label space, at $N = 10$ and ring degree 2^{15} under collective refresh, for the tournament and for the sequential fold it replaces.

AG-News, 5.0×10^4 on DBpedia and 2.0×10^5 on Banking77. The head has Cd parameters, where d is the feature dimension, and in each case fidelity 0.90 costs between three and five times that number of queries. A deployment can therefore set its allowance from C , d and the fidelity it concedes. A copy at that fidelity reproduces the served model's decisions, and published membership attacks against it and against the head in plaintext both sit at chance. Section V of the technical report [18] gives the extraction study in full.

a) Comparison with a released model: A participant holding the model in plaintext has fidelity 1 at zero queries, may read the parameters directly rather than infer them, and is subject to no allowance because there is nothing to meter.

G. Comparison with Model Disclosure

Never disclosing the model costs accuracy. The quantity $A_{\text{dis}} - A_{\text{sel}}$ runs from 0.029 to 0.136 across the five tasks. A disclosed model does not satisfy Functionality 1, so we do not offer it as a configuration of this protocol. Section V of the technical report [18] gives the comparison per task.

H. Scope and Limitations

Three limitations bound the construction. The shared quantity must be linear in the features, the query allowance is an operational control rather than cryptographic hardness, and the guarantees assume semi-honest parties. Section V of the technical report [18] states each one in full.

VI. RELATED WORK

A. Attacks at Training Time and at Inference Time

Federated learning (FL) began as an iterative procedure in which a server repeatedly broadcasts a model, clients train it on their own data, and the server averages the returned updates [4]. An observer of the per-round updates infers membership and unintended properties of another client's data, and the repetition of those updates is what makes the attack strong [5], [6]. A server that chooses the weights it broadcasts does not need inference at all, since crafted weights make clients return verbatim copies of their own inputs [28].

Attacks that see only the final model are consistently weaker, and the field survey draws that separation explicitly [29]. Against models that generalise well they identify few

members, and only at low false-positive rates [30]. A protocol that never sends an update, and that sends anything only once, therefore leaves every adversary with the weaker attack.

B. Protecting Training with Cryptography

Secure aggregation lets the server learn only the sum of the clients' updates, using masks that cancel in the aggregate [31]. It protects one round's sum inside an iterative protocol, and Kerkouche et al. show that the revealed per-round sums themselves leak across rounds [32]. POSEIDON runs encrypted stochastic gradient descent under multiparty CKKS, with activations replaced by polynomial approximations [12]. The approach is iterative and encrypted nonlinearity costs a deep multiplicative circuit with repeated bootstrapping. That cost scales with the network being trained, therefore adapting a pretrained transformer by this route is out of reach.

Several works encrypt only the aggregation step of an otherwise plaintext training loop [33]. All of them remain bound to the iterative protocol, hence the encryption cost is paid on every round and what is protected is a per-round sum rather than a complete contribution. Single-key homomorphic encryption forces the assumption that the server and the clients do not collude [33]. The threshold structure of Section II-A removes that assumption.

Transfer learning itself has been placed under encryption. Several works train a classifier head, a linear or logistic model on frozen features under CKKS [34]. These share our structural niche, but they run an optimiser on ciphertexts, hence multiplicative depth grows with the step count.

C. One-Shot Federated Learning and Differential Privacy

A parallel line collapses the many rounds into a single exchange [8]. Genuinely single-round methods distil the clients' models through a generator or synthesised inputs, or fuse the models directly [9], [10]. These establish that one round can produce a usable model, but they work in plaintext, and each client sends an entire model or a synthetic distillate of its data. A recent method sends no model at all, having each client upload per-class sufficient statistics once [35]. Those statistics travel in plaintext, and per-class totals are exactly what Section III-B keeps encrypted.

The one-shot methods that do address privacy use differential privacy. FedAUXfdp releases client contributions under

a differentially private mechanism after distillation through a pretrained extractor [11], and related approaches protect a diffusion-generated surrogate or a teacher ensemble [36], [37]. These are our natural quantitative peers, and their privacy is lossy, because the noise a meaningful budget requires is added to the very artifact that is released. Several works apply the same trade-off to transfer learning, running noisy gradient descent on a low-rank adapter [38].

D. Federated Fine-Tuning and Task Arithmetic

A recent line federates parameter-efficient fine-tuning, communicating a low-rank adapter rather than a model. FedIT averages the adapters round by round [39], but per-factor averaging is biased. FFA-LoRA instead freezes the randomly initialised down-projection and trains only the up-projection, hence averaging the trained factor is exact [25]. None of this line protects the contributions, which is our subject.

E. Secure Inference

Answering queries from a model that stays encrypted is the subject of secure inference. Several works evaluate a transformer under encryption, and in each case the nonlinearities of attention dominate the cost [40]. The map served here is linear in the features, thus serving a trained map that sits inside the backbone instead would reduce to exactly the problem these systems address.

Two systems sit closer. slytHERin evaluates a network under CKKS with the model held under a multiparty key, and it key-switches the prediction to the querier, which is the serving setting of Section III-C [41]. It evaluates the whole network homomorphically, which is why the models it reports are a twenty-layer and a fifty-layer network on a handwritten-digit task rather than a pretrained encoder. And it returns the prediction vector, hence the querier receives scores, which Section V-F shows is a cheaper target for extraction than a label.

CryptPEFT divides the work as we do. A public frozen backbone runs in plaintext on the client, the client encrypts the resulting features, only the adapter is evaluated under encryption, and the client receives the adapter's output [42]. It is a two-party protocol, the trained unit is the provider's plaintext, and there is no training protocol.

Positioning. Table III places the closest systems on the axes that separate them. What remains unaddressed is a protocol that exposes no training-time quantity, that exchanges anything at all only once, and that never discloses the model even to the parties that built it. Section VI of the technical report [18] surveys the field in full.

VII. CONCLUSION

We presented HE-OFT, a federated fine-tuning protocol in which no party receives the trained result. The design follows from the observation that federated learning's privacy risk is concentrated in the quantities a protocol exposes while the model is being built, and from the further requirement that in some deployments the finished model may not be distributed

TABLE III

THE CLOSEST SYSTEMS ON THE FOUR AXES THIS PAPER TURNS ON, AS EACH PAPER REPORTS THEM. THE PLAINTEXT COLUMN SAYS WHETHER ANY PARTY ENDS UP HOLDING THE TRAINED MODEL IN PLAINTEXT, AND *provider* MEANS ONE PARTY DOES WHILE THE QUERIER DOES NOT.

System	Rounds	Protection	Model in plaintext	Querier receives
DENSE [9]	one	none	yes	the model
GH-OFL [35]	one	none	yes	the model
FedAUXfdp [11]	one	DP	yes	the model
POSEIDON [12]	many	HE	no	the prediction
slytHERin [41]	many	HE	no	scores
CryptPEFT [42]	many	2PC	provider	scores
HE-OFT	one	HE	no	label

either. Each client fine-tunes on a frozen public backbone, retains its adapter locally, and uploads a single encrypted head displacement. The server combines those displacements under multiparty CKKS and never decrypts the result. A quorum of clients returns only the predicted label, addressed to the party that asked.

Withholding the model constrains what may be shared. Since the querying party must form its query from components it can evaluate itself, those components must be public, and the shared trained quantity must therefore attach after the last nonlinearity of the network. Across four text classification tasks and one vision task, the protocol produces a model stronger than any client obtains alone, at a cost of 0.03 to 0.14 accuracy relative to a model that is aggregated and disclosed.

Two directions follow. Robustness to actively malicious participants is outside the present guarantee, and reconciling robust aggregation with an encrypted contribution remains open. Serving a shared quantity that sits inside the backbone, rather than after it, reduces to secure transformer inference.

ACKNOWLEDGMENTS

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

REFERENCES

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *ICLR*, 2022.
- [2] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.
- [3] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 (general data protection regulation)," *Official Journal of the European Union*, L 119, pp. 1–88, 2016.
- [4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.
- [5] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019.
- [6] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE S&P*, 2019.

- [7] G. Zhu, D. Li, H. Gu, Y. Yao, L. Fan, and Y. Han, "FedMIA: An effective membership inference attack exploiting "all for one" principle in federated learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 20 643–20 653.
- [8] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019.
- [9] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, 2022.
- [10] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.
- [11] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.
- [12] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.
- [13] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 (artificial intelligence act)," Official Journal of the European Union, L, 2024/1689, 2024.
- [14] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017.
- [15] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.
- [16] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv:1907.11692*, 2019.
- [17] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.
- [18] H. I. Kanpak, S. Sav, and A. Küpçü, "HE-OFT: Privacy-preserving one-shot federated fine-tuning under homomorphic encryption (technical report)," *arXiv preprint arXiv:XXXX.XXXXX*, 2026.
- [19] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *USENIX Security*, 2016.
- [20] N. Carlini, D. Paleka, K. D. Dvijotham, T. Steinke, J. Hayase, A. F. Cooper, K. Lee, M. Jagielski, M. Nasr, A. Conmy, I. Yona, E. Wallace, D. Rolnick, and F. Tramèr, "Stealing part of a production language model," in *ICML*, 2024.
- [21] Y. Chen, X. Dong, J. Guo, Y. Shen, A. Wang, and X. Wang, "Hard-label cryptanalytic extraction of neural network models," in *ASIACRYPT*, 2024.
- [22] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *ASIACRYPT*, 2020.
- [23] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," in *IEEE S&P*, 2023.
- [24] F. Karakoç, A. Küpçü, and M. Önen, "Fault tolerant and malicious secure federated learning," in *CANS*, 2024.
- [25] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving LoRA in privacy-preserving federated learning," in *ICLR*, 2024.
- [26] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.
- [27] C. Mouchet, J.-P. Bossuat, J. R. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.
- [28] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," in *IEEE EuroS&P*, 2023.
- [29] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [30] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *IEEE S&P*, 2022.
- [31] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017.
- [32] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*, 2023.
- [33] J. Liu, L. Yan, B. Li, L. Yu, and C. Shen, "SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA," in *ICLR*, 2026.
- [34] S. Lee, G. Lee, J. W. Kim, J. Shin, and M.-K. Lee, "HETAL: Efficient privacy-preserving transfer learning with homomorphic encryption," in *ICML*, 2023.
- [35] F. Turazza, M. Picone, and M. Mamei, "The Gaussian-Head OFL family: One-shot federated learning from client global statistics," in *ICLR*, 2026.
- [36] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025.
- [37] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021.
- [38] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, "Differentially private low-rank adaptation of large language model using federated learning," *ACM TMIS*, vol. 16, no. 2, pp. 1–24, 2025.
- [39] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, Y. Zhou, G. Wang, and Y. Chen, "Towards building the federated GPT: Federated instruction tuning," in *IEEE ICASSP*, 2024.
- [40] J. Zhang, J. Liu, X. Yang, Y. Wang, K. Chen, X. Hou, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," in *NDSS*, 2025.
- [41] F. Intoci, S. Sav, A. Pyrgelis, J.-P. Bossuat, J. R. Troncoso-Pastoriza, and J.-P. Hubaux, "slytHERin: An agile framework for encrypted deep neural network inference," in *Cloud S&P*, 2023.
- [42] S. Xia, W. Wang, Z. Wang, Y. Zhang, Y. Jin, D. Meng, and R. Hou, "CryptPEFT: Efficient and private neural network inference via parameter-efficient fine-tuning," in *NDSS*, 2026.



HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.



Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.



Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.